

Version control and git

so what is version control? And why we use it

So basically, Version Control is a system that records the changes made to a file or a set of files over time. Because of this, we can go back and see any specific version of our project whenever we need it.

If we take an example we will know this in detail

Suppose we are working on a project and everything is working perfectly. After that, we make some changes in the code. Maybe we add a new feature or modify some existing functionality.

After that if our project starts giving us error so by the help of this we can go to the older version of our file

so this is how version control helps us
the feature of the version control our

We can keep the track of changes we made in our project
then we can compare it to the older version of our project

To work with the multiple people

so if we explain this with some example so we should say like three people work on the same project but one is working on the login feature the other is working on the payment feature and then the other is working on the bugs so if any of these developers directly changes the same file without any proper system it can create a problem for everyone

To find who made the changes

Before git there were different version control systems like cvs then svn(subversion) and bitkeeper

But the problem they were getting is whenever there are very large projects and many developers are working on the same project version control becomes more complex to use

So git was created for the linux kernel project first as it was very large and complex project

So basically for some time linux kernel community uses the bitkeeper

So then what happened is there were problems like relationship and licensing arrangements by using the bitkeeper

then the solution comes in that is distributed development in which a developer should work locally then make changes then create commits and then maintain history without always depending on the one central server

So git was created in 2005

So the main goal is to make the vcs system that is fast distributed then reliable efficient good suitable for the large projects

So it was not created randomly

in vcs there was one central server but if we talk about the dvcs there are different local repositories for different developers which solve a huge problem

Now we are coming to the main part of the git
so git is designed to be fast
these developers perform git operations all the time

Git status

Git log

Git branch

Git diff

So it was designed so that many operations could be handled locally

GIT provides features like

Branching then merging then commit history and distributed repositories

Git is used in different projects

Like software development Web development data engineering devops machine learning

What is git ?

Git is a distributed version control system

it helps us to track and manage the changes we have made in our project time to time

Why do we use git

Git provides a powerful way of performing the version control

Important features in git

Branching experiment

Can git work without GitHub

Yes, git and GitHub are not the same thing

We can use git completely on our local computer

Eg

Git init we can create a repository

Git add. We can stage our changes

Git commit -m "initial changes"

So now we will understand what is GitHub ?

GitHub is an online platform that helps us to host and share git repositories

GitHub becomes useful for Storing repositories online, Collaborating with teams, Sharing projects, Creating Pull Requests and Reviewing code

Components in github

Working directories

Staging area

Local repository

Remote repository

Important git commands

Git init - used to create a Git repository.

Git status – used to check the current status of the repository

Git add – use to move changes to the staging area

Git commit – used to save staged changes as a commit

Git log – used to see commit history

Git branch - used to work with the branch

Git merge -Used to combine changes from different branches

Git Ignore - Sometimes there are files that we don't want Git to track

GitHub Repository

A repository is basically the project where Git stores and manages our files and history.

When this repository is stored on our computer, it is called - Local Repository

When it is hosted on GitHub, it becomes - Remote Repository

Introduction to data engineering

What is Data Engineering and Why is it Important

Data engineering is basically the process of taking raw data and converting it into structured, useful data that people can actually use.

Why Do We Need Data Engineers?

So the amount of data being generated today is huge Managing this much data is not easy.

The Core Data Engineering Workflow

Data Production - where the data is generated

Data Transformation- when we know that the data is raw and we have to clean then normalize it

Data Serving - After we have transformed the data, we need to serve it this means making it available to the people who need it.

OLTP vs OLAP

OLTP is for real-time transactions - like when someone buys something.

OLAP is for analytics and reporting - like when we want to see sales trends.

ETL and Pipelines

ETL means Extract, Transform, Load.

We take data from source, clean it, then load it to destination.

Pipelines automate this whole process.

Incremental Loading means we only process new data, not everything again.

This saves time and money.

Dimensional Modeling

Fact Table stores numbers like sales amount, quantity.

Dimension Table stores descriptions like product name, customer name.

Star Schema has one fact table connected to many dimension tables.

Snowflake Schema is more complex where dimensions are broken further.

Star schema is preferred because it's simple and fast.

Data Lake and Lakehouse

Data Warehouse stores structured data, schema-on-write. Cost is higher.

Data Lake stores everything, schema-on-read. Cost is cheaper.

Lakehouse combines both. Data stored cheaply in lake. Metadata layer adds schema on top.

Supports ACID transactions and schema evolution.

Big Data

Big data needs multiple machines working together. This is called a cluster.

Apache Spark is the main framework. It processes data in-memory.

Driver orchestrates the work. Worker nodes do the actual processing.

Apache Kafka handles real-time streaming.

Apache Airflow schedules and manages pipelines.